Introduction to eBPF Load-Acquire and Store-Release Instructions

Peilin Ye (Google)

東京 2025
LINUX PLUMBERS CONFERENCE
TOKYO, JAPAN / DEC. 11-13, 2025

```c
int64_t foo = __sync_fetch_and_or(ptr, 0);
```

```
// Eventually this could be a load-acquire...
int64_t foo = __sync_fetch_and_or(ptr, 0);
```

What if I...

```
short foo(short *ptr) {
    return __atomic_load_n(ptr, __ATOMIC_ACQUIRE);
}
```

```
$ clang-19 --target=bpf -mcpu=v4 -g -c -O2 demo.bpf.c > /dev/null
```

```
$ clang-19 --target=bpf -mcpu=v4 -g -c -O2 demo.bpf.c > /dev/null
fatal error: error in backend: Cannot select: 0x55bf614cc030: i32,ch = AtomicLoad<(load acquire (s16) from %ir.0), zext from i16> 0x55bf6146d488, 0x55bf614cbd90, demo.bpf.c:2:9
  0x55bf614cbd90: i64,ch = CopyFromReg 0x55bf6146d488, Register:i64 %0
    0x55bf614cbd20: i64 = Register %0
In function: foo
PLEASE submit a bug report to https://github.com/llvm/llvm-project/issues/ and include the crash backtrace, preprocessed source, and associated run script.
Stack dump:
0.	Program arguments: clang-19 --target=bpf -mcpu=v4 -g -c -O2 demo.bpf.c
1.	<eof> parser at end of file
2.	Code generation
3.	Running pass 'Function Pass Manager' on module 'demo.bpf.c'.
4.	Running pass 'BPF DAG->DAG Pattern Instruction Selection' on function '@foo'
 #0 0x00007f5217eb067a llvm::sys::PrintStackTrace(llvm::raw_ostream&, int) (/usr/lib/llvm-19/bin/../lib/libLLVM.so.19.1+0xeb067a)
 #1 0x00007f5217eae224 llvm::sys::RunSignalHandlers() (/usr/lib/llvm-19/bin/../lib/libLLVM.so.19.1+0xeae224)
 #2 0x00007f5217df3f3e (/usr/lib/llvm-19/bin/../lib/libLLVM.so.19.1+0xdf3f3e)
 #3 0x00007f5217df3eff (/usr/lib/llvm-19/bin/../lib/libLLVM.so.19.1+0xdf3eff)
 #4 0x00007f5217eaa6fb llvm::sys::Process::Exit(int, bool) (/usr/lib/llvm-19/bin/../lib/libLLVM.so.19.1+0xeaa6fb)
 #5 0x000055bf2d6cf087 (/usr/lib/llvm-19/bin/clang+0x12087)
 #6 0x00007f5217e02d8d llvm::report_fatal_error(llvm::Twine const&, bool) (/usr/lib/llvm-19/bin/../lib/libLLVM.so.19.1+0xe02d8d)
 #7 0x00007f521876ab27 (/usr/lib/llvm-19/bin/../lib/libLLVM.so.19.1+0x176ab27)
 #8 0x00007f521876a028 llvm::SelectionDAGISel::SelectCodeCommon(llvm::SDNode*, unsigned char const*, unsigned int) (/usr/lib/llvm-19/bin/../lib/libLLVM.so.19.1+0x176a028)
 #9 0x00007f521a5e1a99 (/usr/lib/llvm-19/bin/../lib/libLLVM.so.19.1+0x35e1a99)
#10 0x00007f521876159c llvm::SelectionDAGISel::DoInstructionSelection() (/usr/lib/llvm-19/bin/../lib/libLLVM.so.19.1+0x176159c)
#11 0x00007f5218760cdc llvm::SelectionDAGISel::CodeGenAndEmitDAG() (/usr/lib/llvm-19/bin/../lib/libLLVM.so.19.1+0x1760cdc)
#12 0x00007f521875fa70 llvm::SelectionDAGISel::SelectAllBasicBlocks(llvm::Function const&) (/usr/lib/llvm-19/bin/../lib/libLLVM.so.19.1+0x175fa70)
#13 0x00007f521875e372 llvm::SelectionDAGISel::runOnMachineFunction(llvm::MachineFunction&) (/usr/lib/llvm-19/bin/../lib/libLLVM.so.19.1+0x175e372)
#14 0x00007f521875ce5e llvm::SelectionDAGISelLegacy::runOnMachineFunction(llvm::MachineFunction&) (/usr/lib/llvm-19/bin/../lib/libLLVM.so.19.1+0x175ce5e)
#15 0x00007f521829de28 llvm::MachineFunctionPass::runOnFunction(llvm::Function&) (/usr/lib/llvm-19/bin/../lib/libLLVM.so.19.1+0x129de28)
#16 0x00007f5218004424 llvm::FPPassManager::runOnFunction(llvm::Function&) (/usr/lib/llvm-19/bin/../lib/libLLVM.so.19.1+0x1004424)
#17 0x00007f5218009f33 llvm::FPPassManager::runOnModule(llvm::Module&) (/usr/lib/llvm-19/bin/../lib/libLLVM.so.19.1+0x1009f33)
#18 0x00007f5218004b7e llvm::legacy::PassManagerImpl::run(llvm::Module&) (/usr/lib/llvm-19/bin/../lib/libLLVM.so.19.1+0x1004b7e)
#19 0x00007f5220c7d0e8 clang::EmitBackendOutput(clang::DiagnosticsEngine&, clang::HeaderSearchOptions const&, clang::CodeGenOptions const&, clang::TargetOptions const&, clang::LangOptions const&,
llvm::StringRef, llvm::Module*, clang::BackendAction, llvm::IntrusiveRefCntPtr<llvm::vfs::FileSystem>, std::unique_ptr<llvm::raw_pwrite_stream, std::default_delete<llvm::raw_pwrite_stream>>,
clang::BackendConsumer*) (/usr/lib/llvm-19/bin/../lib/libclang-cpp.so.19.1+0x1e7d0e8)
#20 0x00007f522102a551 clang::BackendConsumer::HandleTranslationUnit(clang::ASTContext&) (/usr/lib/llvm-19/bin/../lib/libclang-cpp.so.19.1+0x222a551)
#21 0x00007f521fa344e9 clang::ParseAST(clang::Sema&, bool, bool) (/usr/lib/llvm-19/bin/../lib/libclang-cpp.so.19.1+0xc344e9)
#22 0x00007f5221b47789 clang::FrontendAction::Execute() (/usr/lib/llvm-19/bin/../lib/libclang-cpp.so.19.1+0x2d47789)
#23 0x00007f5221ab6084 clang::CompilerInstance::ExecuteAction(clang::FrontendAction&) (/usr/lib/llvm-19/bin/../lib/libclang-cpp.so.19.1+0x2cb6084)
#24 0x00007f5221bc5c0e clang::ExecuteCompilerInvocation(clang::CompilerInstance*) (/usr/lib/llvm-19/bin/../lib/libclang-cpp.so.19.1+0x2dc5c0e)
#25 0x000055bf2d6cec61 cc1_main(llvm::ArrayRef<char const*>, char const*, void*) (/usr/lib/llvm-19/bin/clang+0x11c61)
```

tldr

BPF load-acquire/store-release now available!

- **LLVM 21:**
  - via compiler built-ins, e.g.:
    - `return __atomic_load_n(ptr, __ATOMIC_ACQUIRE);`
    - `__atomic_store_n(ptr, val, __ATOMIC_RELEASE);`
  - or as inline asm
- **Kernel support:**
  - **6.15:** ARM64 & x86_64
  - **6.16:** riscv64 (Andrea Parri)
  - **6.17:** powerpc64 (Puranjay Mohan)

Agenda

- **WHY:** A refresher on load-acquire/store-release
- **WHAT:** Encodings, usage, and more
- **HOW:** From demo.bpf.c to native instructions
- Future steps
- Questions/comments

1: A refresher on load-acquire/store-release

```
P0(int *buf, int *flag) // Producer
{
    WRITE_ONCE(*buf, 1);
    WRITE_ONCE(*flag, 1);
}

P1(int *buf, int *flag) // Consumer
{
    int r0;
    int r1;

    r0 = READ_ONCE(*flag);
    r1 = READ_ONCE(*buf);
}
```

MP+poonceonces.litmus

- Linux Kernel Memory Model **(LKMM)** litmus test
- **herd7**: reports which **(r0, r1)** outcomes are allowed

$** cd **$KERNEL_SOURCE/tools/memory-model/
$ herd7 -conf linux-kernel.cfg
litmus-tests/MP+poonceonces.litmus

```
P0(int *buf, int *flag) // Producer
{
    WRITE_ONCE(*buf, 1);
    WRITE_ONCE(*flag, 1);
}

P1(int *buf, int *flag) // Consumer
{
    int r0;
    int r1;

    r0 = READ_ONCE(*flag);
    r1 = READ_ONCE(*buf);
}
```

MP+poonceonces.litmus

- One possible but counter-intuitive result:

{r0: 1, r1: 0}

"**flag** says data is ready, but **buf** contains no data"

Conceptually, it's ***as if*** the producer published **flag** too early, and/or the consumer consumed **buf** too early

Memory barriers

- Conceptually, a way to tell CPU “don’t let certain kinds of reordering happen around this point”
- **LL**, **LS**, **SL** and **SS**
  - **L** (Load); **S** (Store)
  - e.g., if my program is **S; L;** then “keep **SL** order” means that other threads must not be able to observe it as if **L** happened *before* **S**
- A **full barrier** is the strongest one **(LL + LS + SL + SS)**
  - e.g., **__sync_fetch_and_or()** implies a full barrier

- **Store-release (LS, SS)**: ensures the store to **buf** is ordered before the store to **flag**
- **Load-acquire (LL, LS)**: prevents loading **buf** before loading **flag**
- The **{r0: 1, r1: 0}** combination is no longer possible

```
P0(int *buf, int *flag) // Producer
{
    WRITE_ONCE(*buf, 1);
    WRITE_ONCE(*flag, 1);
    smp_store_release(flag, 1);
}

P1(int *buf, int *flag) // Consumer
{
    int r0;
    int r1;

    r0 = READ_ONCE(*flag);
    r0 = smp_load_acquire(flag);
    r1 = READ_ONCE(*buf);
}
```

MP+pooncerelease+poacquireonce.litmus

$** cd **$KERNEL_SOURCE/tools/memory-model/
$ herd7 -conf linux-kernel.cfg
litmus-tests/MP+pooncerelease+poacquireonce.litmus

Motivation

- When possible, use load-acquire/store-release instead of defaulting to a full barrier (e.g., the **__sync_fetch_and_or()** example) for better performance/readability
- Became more interesting as we were looking at running our code on ARM64, as it has a more relaxed memory model

2: Encodings, usage and more

Encodings

- **BPF_STX | BPF_ATOMIC** instructions, with **imm** (the **“immediate”** field) set to:
  - For load-acquire: **BPF_LOAD_ACQ (0x100)**
  - For store-release: **BPF_STORE_REL (0x110)**

Encodings, cont

```
cb 10 00 00 00 01 00 00  w0 = load_acquire((u16 *)(r1 + 0x0))
```

Example (16-bit load-acquire):

- opcode (0xcb/**0b**11001011):
  - class (0b011): STX
  - mode (0b110): ATOMIC
  - sz (0b01): H (16-bit)
- imm (0x00000100): LOAD_ACQ

Differences with existing BPF_ATOMIC instructions

- Existing **BPF_ATOMIC** instructions perform **Read-Modify-Write (RMW)** operations e.g. **fetch-and-add**
- Load-acquire/store-release support all size modifiers **(8-, 16-, 32- and 64-bit)**, while the existing ones only support 32- and 64-bit

Usage

- via the **__atomic_load[_n]()** and **__atomic_store[_n]()** built-ins:

```
return __atomic_load_n(ptr, __ATOMIC_ACQUIRE);
```

```
__atomic_store_n(ptr, val, __ATOMIC_RELEASE);
```

- make sure to use the **-mcpu=v4** flag

Usage, other __ATOMIC_* flags

- __ATOMIC_RELAXED
  - __atomic_load[_n](): plain BPF_LDX | BPF_MEM load
    - zero-extending
  - __atomic_store[_n](): plain BPF_STX | BPF_MEM store
- __ATOMIC_CONSUME
  - effectively an alias to __ATOMIC_ACQUIRE

Usage, other __ATOMIC_* flags, cont

- __ATOMIC_ACQ_REL
  - illegal for __atomic_load[_n]() and __atomic_store[_n]() built-ins
- __ATOMIC_SEQ_CST (sequentially consistent)
  - unsupported yet:

```
seq_cst.bpf.c:2:9: error: sequentially consistent (seq_cst)
atomic load/store is not supported
    2 |         return __atomic_load_n(ptr, __ATOMIC_SEQ_CST);
      |                ^
```

Inline assembly

```
long load_acquire_64(long *ptr)
{
  long ret;
  asm volatile("%0 = load_acquire((u64 *)(%1 + 0x0))" :
               "=r"(ret) : "r"(ptr) : "memory");
  return ret;
}
```

Inline assembly, cont

```
asm volatile("store_release((u16 *)(%0 + 0x0), %1)" :
             : "r"(ptr), "w"(val) : "memory");
```

- Remember to use the **“w”** (instead of **“r”**) input/output constraint when working with **8-**, **16-** and **32-bit** values

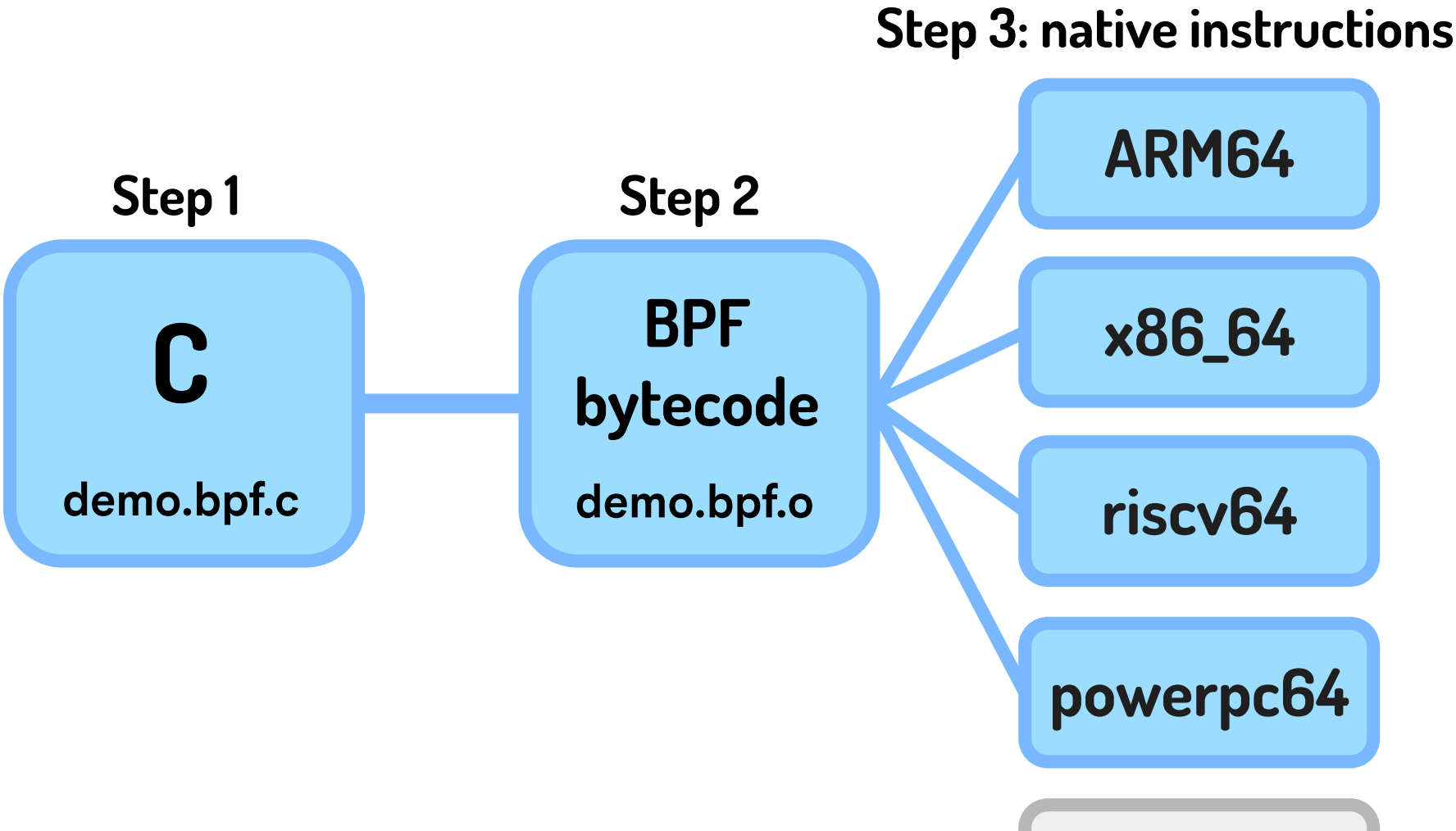
Miscellaneous

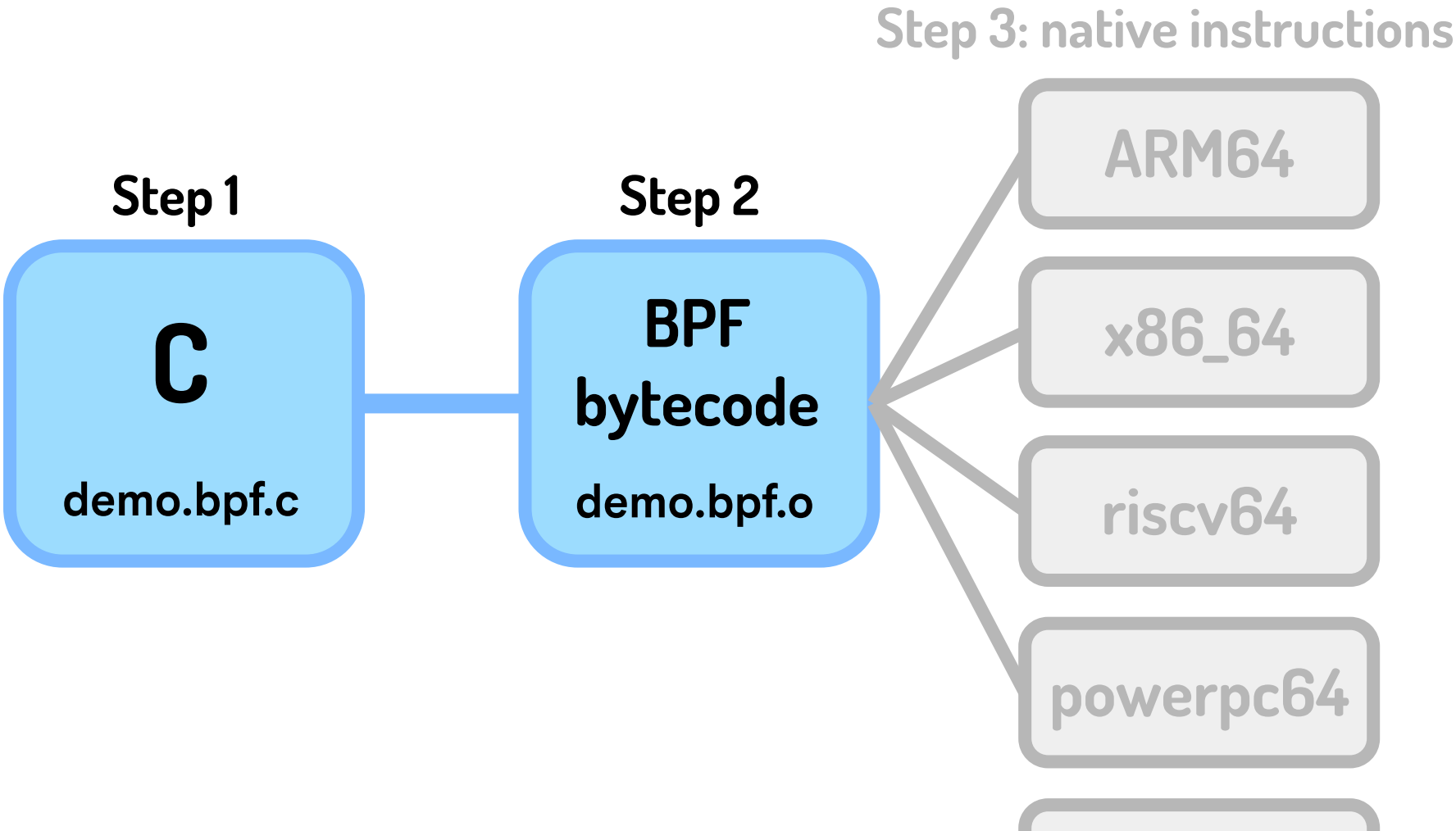
- Arena access is supported (currently for ARM64, x86_64 and riscv64)
- **#ifdef __BPF_FEATURE_LOAD_ACQ_STORE_REL** to detect if the compiler supports BPF load-acquire/store-release

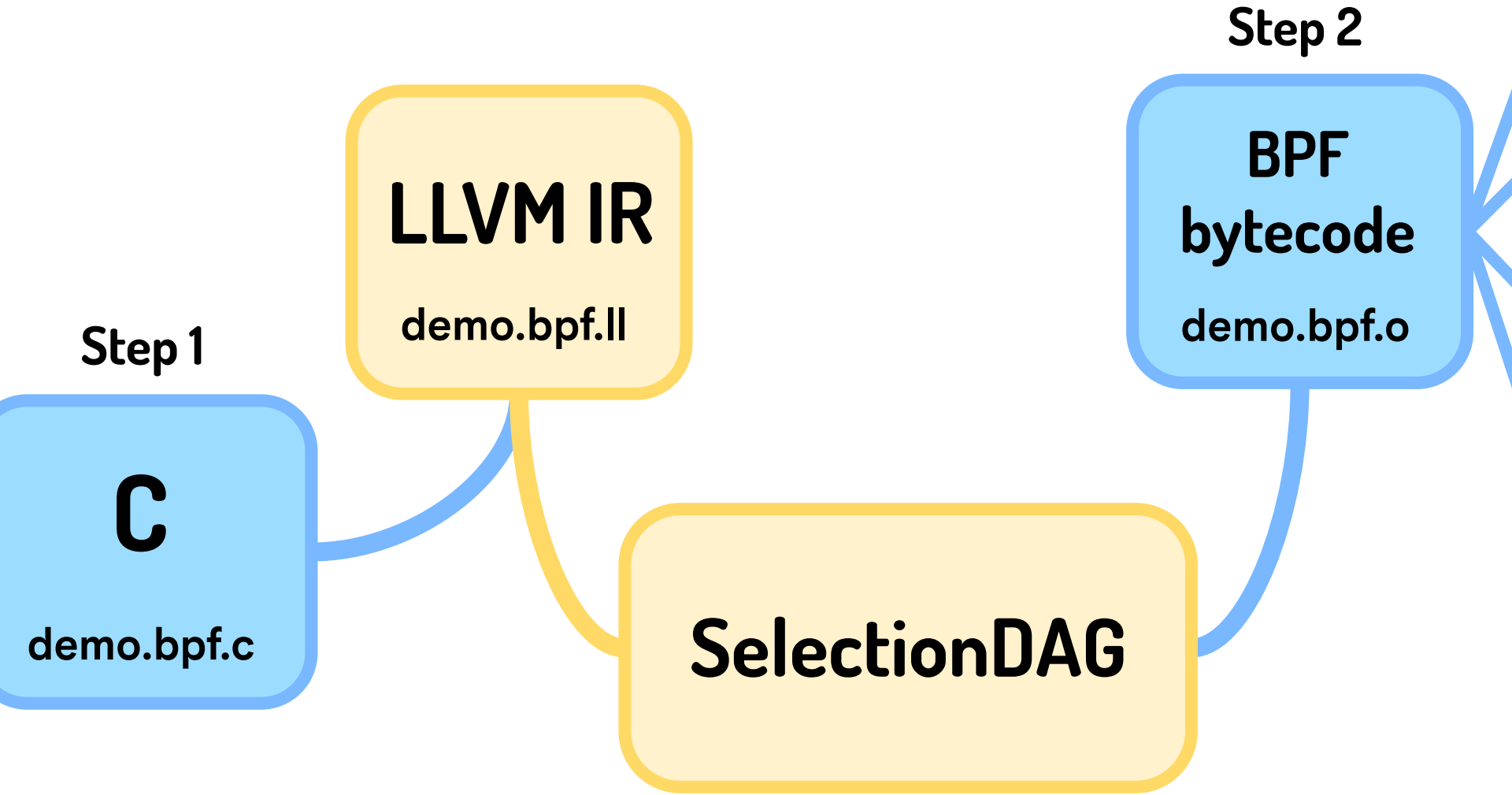
3: From demo.bpf.c to native instructions

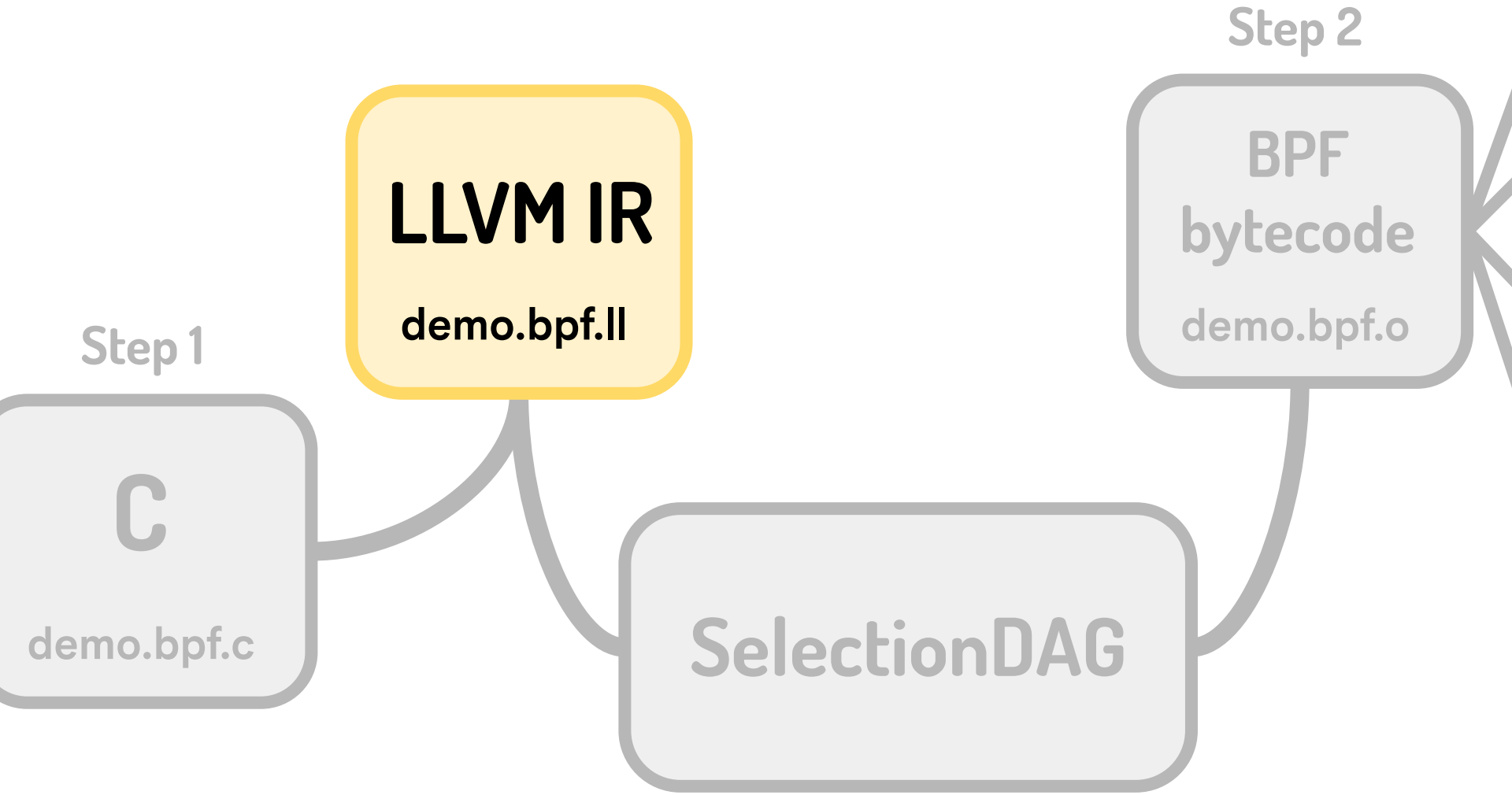
demo.bpf.c: 16-bit BPF load-acquire

```
short foo(short *ptr) {
    return __atomic_load_n(ptr, __ATOMIC_ACQUIRE);
}
```


Step 3: native instructions
Step 1
C
demo.bpf.c
Step 2
BPF
bytecode
demo.bpf.o
ARM64
x86_64
riscv64
powerpc64


Step 3: native instructions
Step 1
C
demo.bpf.c
Step 2
BPF
bytecode
demo.bpf.o
ARM64
x86_64
riscv64
powerpc64

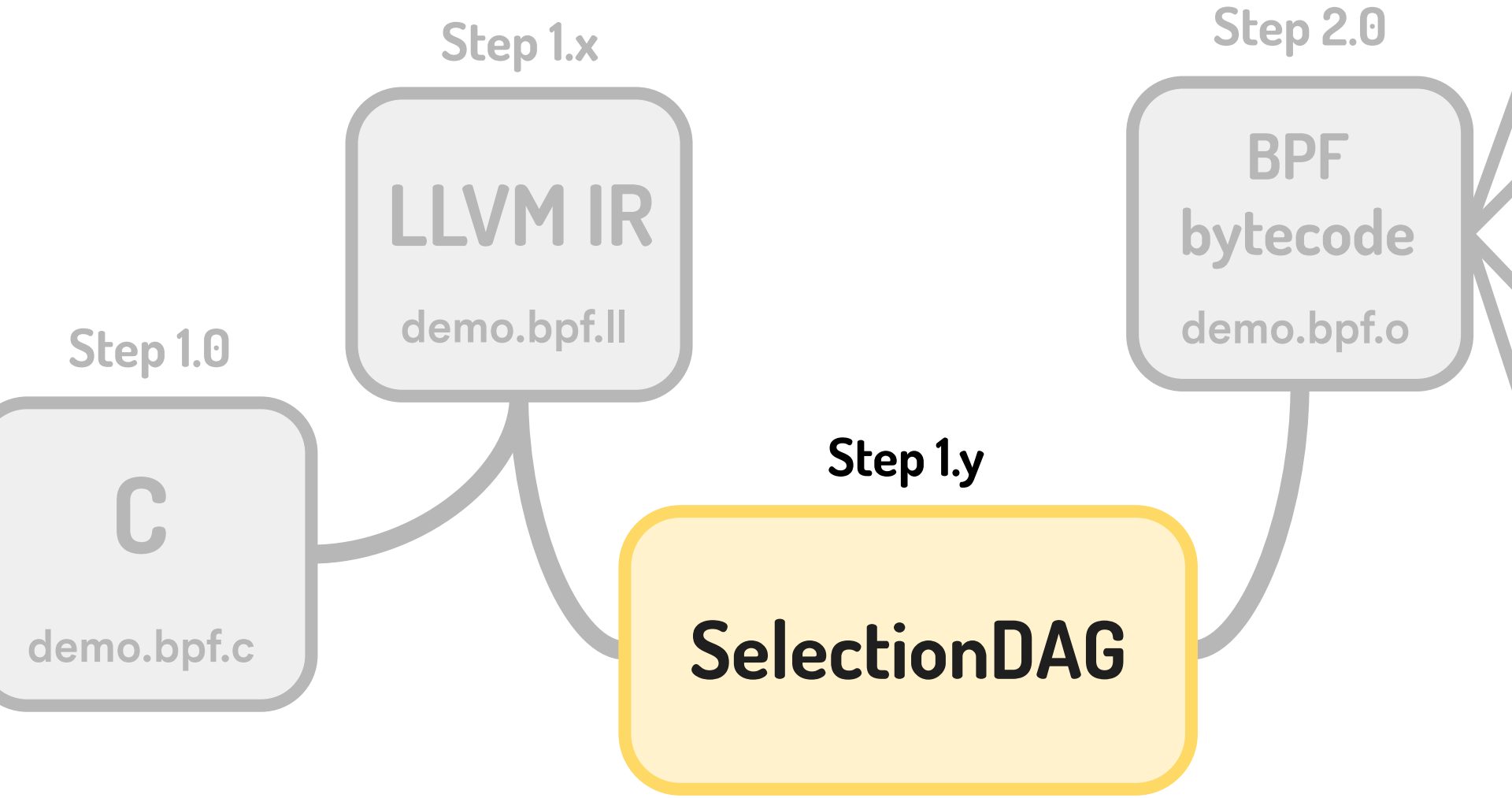

Step 2
LLVM IR
demo.bpf.ll
BPF
bytecode
demo.bpf.o
Step 1
C
demo.bpf.c
SelectionDAG


Step 2
LLVM IR
demo.bpf.ll
BPF
bytecode
demo.bpf.o
Step 1
C
demo.bpf.c
SelectionDAG

demo.bpf.ll: LLVM Intermediate Representation

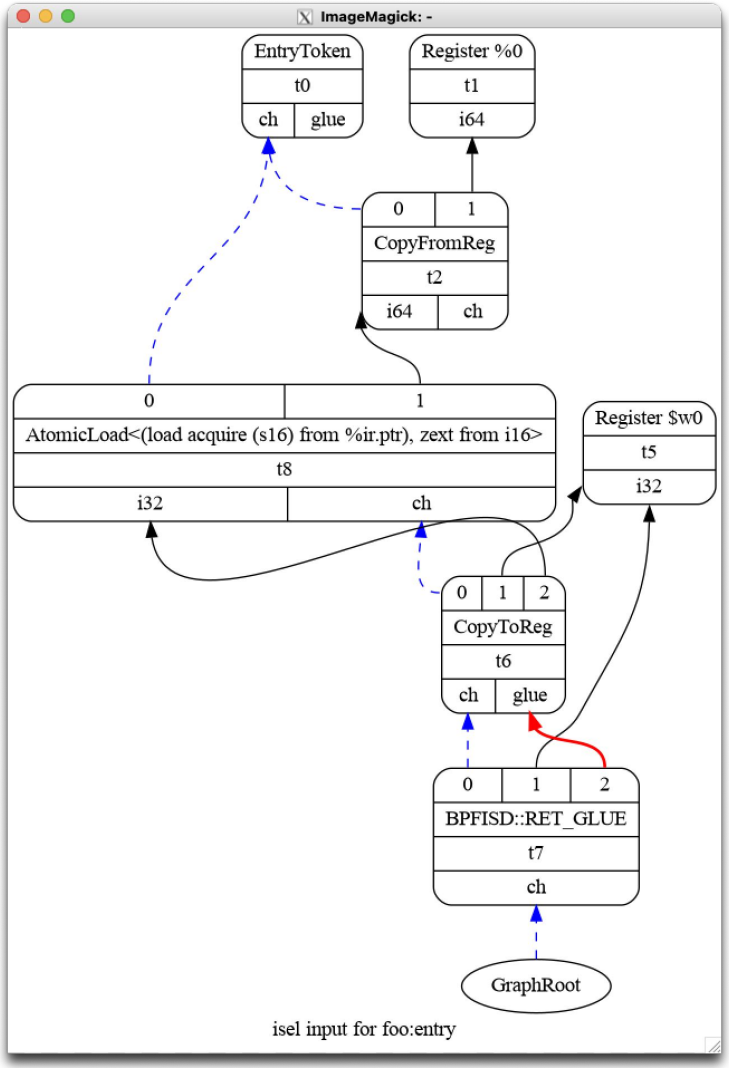
```
...
entry:
  %0 = load atomic i16, ptr %ptr acquire, align 2
  ret i16 %0
}
...
```

$ clang -S **-emit-llvm** -O2 demo.bpf.c


Step 1.x
LLVM IR
demo.bpf.ll
Step 2.0
BPF
bytecode
demo.bpf.o
Step 1.0
C
demo.bpf.c
Step 1.y
SelectionDAG

SelectionDAG

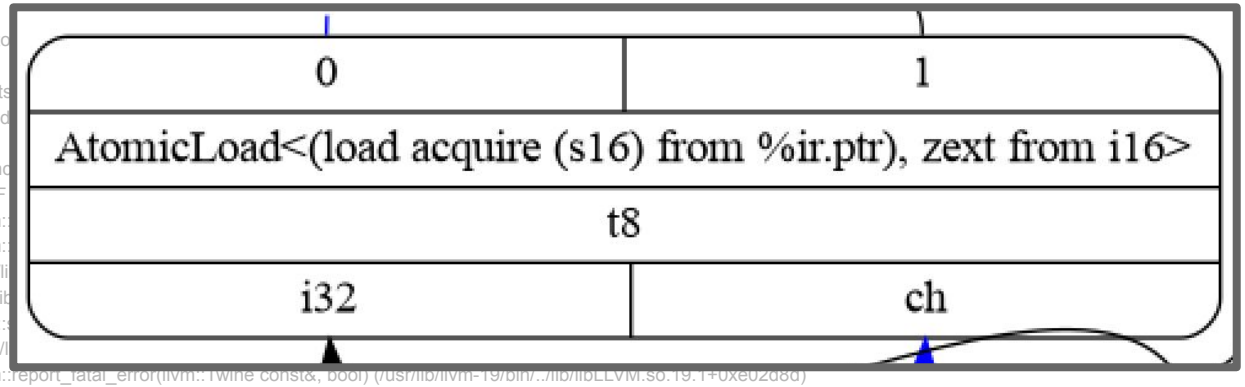
- **SelectionDAG** representation of the demo program, right before we (try to) turn it into another DAG of target **(i.e. BPF)** instructions



$ clang --target=bpf -mcpu=v4 -mllvm **-view-isel-dags** -O2 -g -c -o demo.bpf.o demo.bpf.c

$** dot -Tpng **$DOT_FILE | display

```
$ clang-19 --target=bpf -mcpu=v4 -g -c -O2 demo.bpf.c > /dev/null
fatal error: error in backend: Cannot select: 0x55bf614cc030: i32,ch = AtomicLoad<(load acquire (s16) from %ir.0), zext from i16> 0x55bf6146d488, 0x55bf614cbd90, demo.bpf.c:2:9
  0x55bf614cbd90: i64,ch = CopyFromReg 0x55bf6146d488, Register:i64 %0
    0x55bf614cbd20: i64 = Register %0
In function: foo
PLEASE submit a bug report to
Stack dump:
0.      Program arguments
1.      <eof> parser at end
2.      Code generation
3.      Running pass 'Func
4.      Running pass 'BPF
 #0 0x00007f5217eb067a llvm::
 #1 0x00007f5217eae224 llvm::
 #2 0x00007f5217df3f3e (/usr/li
 #3 0x00007f5217df3eff (/usr/lib
 #4 0x00007f5217eaa6fb llvm::
 #5 0x000055bf2d6cf087 (/usr/l
 #6 0x00007f5217e02d8d llvm::report_fatal_error(llvm::Twine const&, bool) (/usr/lib/llvm-19/bin/../lib/libLLVM.so.19.1+0xe02d8d)
 #7 0x00007f521876ab27 (/usr/lib/llvm-19/bin/../lib/libLLVM.so.19.1+0x176ab27)
 #8 0x00007f521876a028 llvm::SelectionDAGISel::SelectCodeCommon(llvm::SDNode*, unsigned char const*, unsigned int) (/usr/lib/llvm-19/bin/../lib/libLLVM.so.19.1+0x176a028)
 #9 0x00007f521a5e1a99 (/usr/lib/llvm-19/bin/../lib/libLLVM.so.19.1+0x35e1a99)
#10 0x00007f521876159c llvm::SelectionDAGISel::DoInstructionSelection() (/usr/lib/llvm-19/bin/../lib/libLLVM.so.19.1+0x176159c)
#11 0x00007f521
#12 0x00007f521
#13 0x00007f521
#14 0x00007f521
#15 0x00007f521
#16 0x00007f521
#17 0x00007f521
#18 0x00007f521
#19 0x00007f522                                                                                                                    Options const&,
llvm::StringRef, l
clang::BackendConsumer*) (/usr/lib/llvm-19/bin/../lib/libclang-cpp.so.19.1+0x1e7d0e8)
#20 0x00007f522102a551 clang::BackendConsumer::HandleTranslationUnit(clang::ASTContext&) (/usr/lib/llvm-19/bin/../lib/libclang-cpp.so.19.1+0x222a551)
#21 0x00007f521fa344e9 clang::ParseAST(clang::Sema&, bool, bool) (/usr/lib/llvm-19/bin/../lib/libclang-cpp.so.19.1+0xc344e9)
#22 0x00007f5221b47789 clang::FrontendAction::Execute() (/usr/lib/llvm-19/bin/../lib/libclang-cpp.so.19.1+0x2d47789)
#23 0x00007f5221ab6084 clang::CompilerInstance::ExecuteAction(clang::FrontendAction&) (/usr/lib/llvm-19/bin/../lib/libclang-cpp.so.19.1+0x2cb6084)
#24 0x00007f5221bc5c0e clang::ExecuteCompilerInvocation(clang::CompilerInstance*) (/usr/lib/llvm-19/bin/../lib/libclang-cpp.so.19.1+0x2dc5c0e)
#25 0x000055bf2d6cec61 cc1_main(llvm::ArrayRef<char const*>, char const*, void*) (/usr/lib/llvm-19/bin/clang+0x11c61)
```

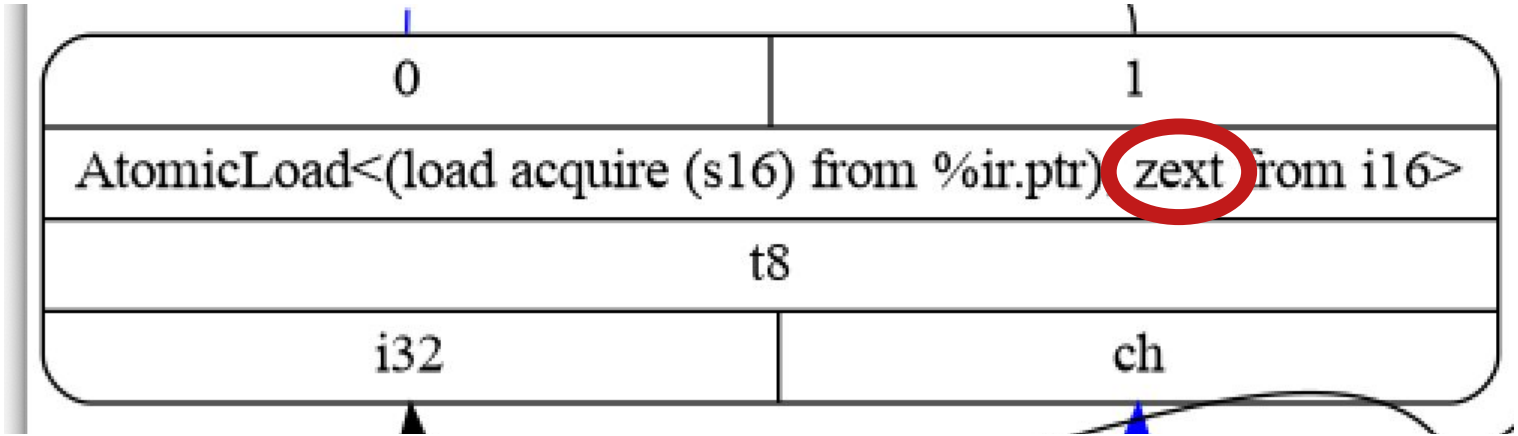


fatal error: error in backend: Cannot select: <...>: i32,ch = AtomicLoad<(load acquire (s16) from %ir.0), zext from i16>

SelectionDAG, cont

- LLVM tries to translate this (acquiring) **AtomicLoad** node by looking up an auto-generated table called **MatcherTable**, but nothing matches
- **MatcherTable** is generated from **llvm/lib/Target/BPF/*.td** (**.td** stands for **T**arget **D**escription) files, via an infrastructure called **TableGen**
- We modify those **.td** files to teach LLVM how to translate an acquiring **AtomicLoad**

Sign extension

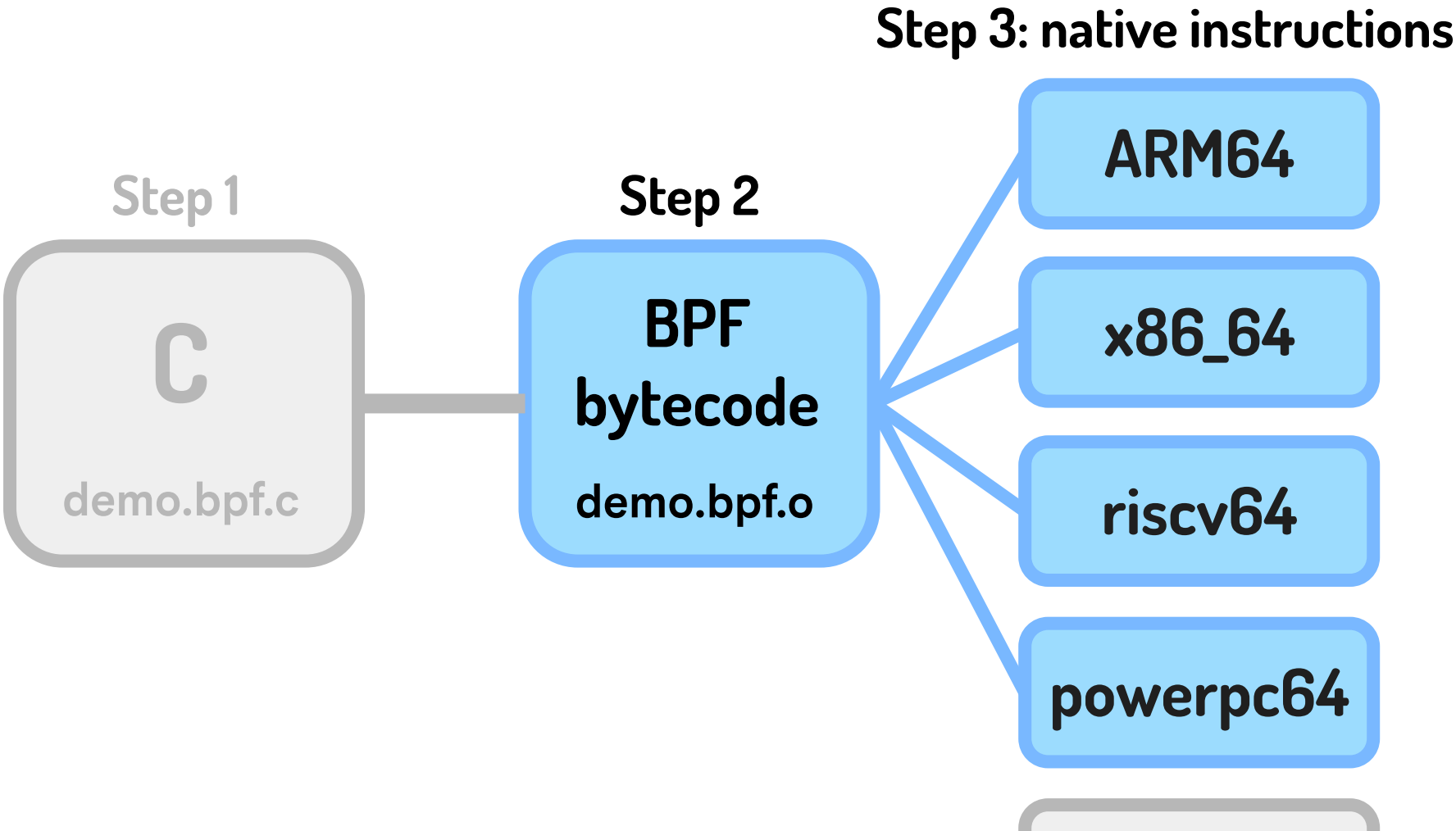


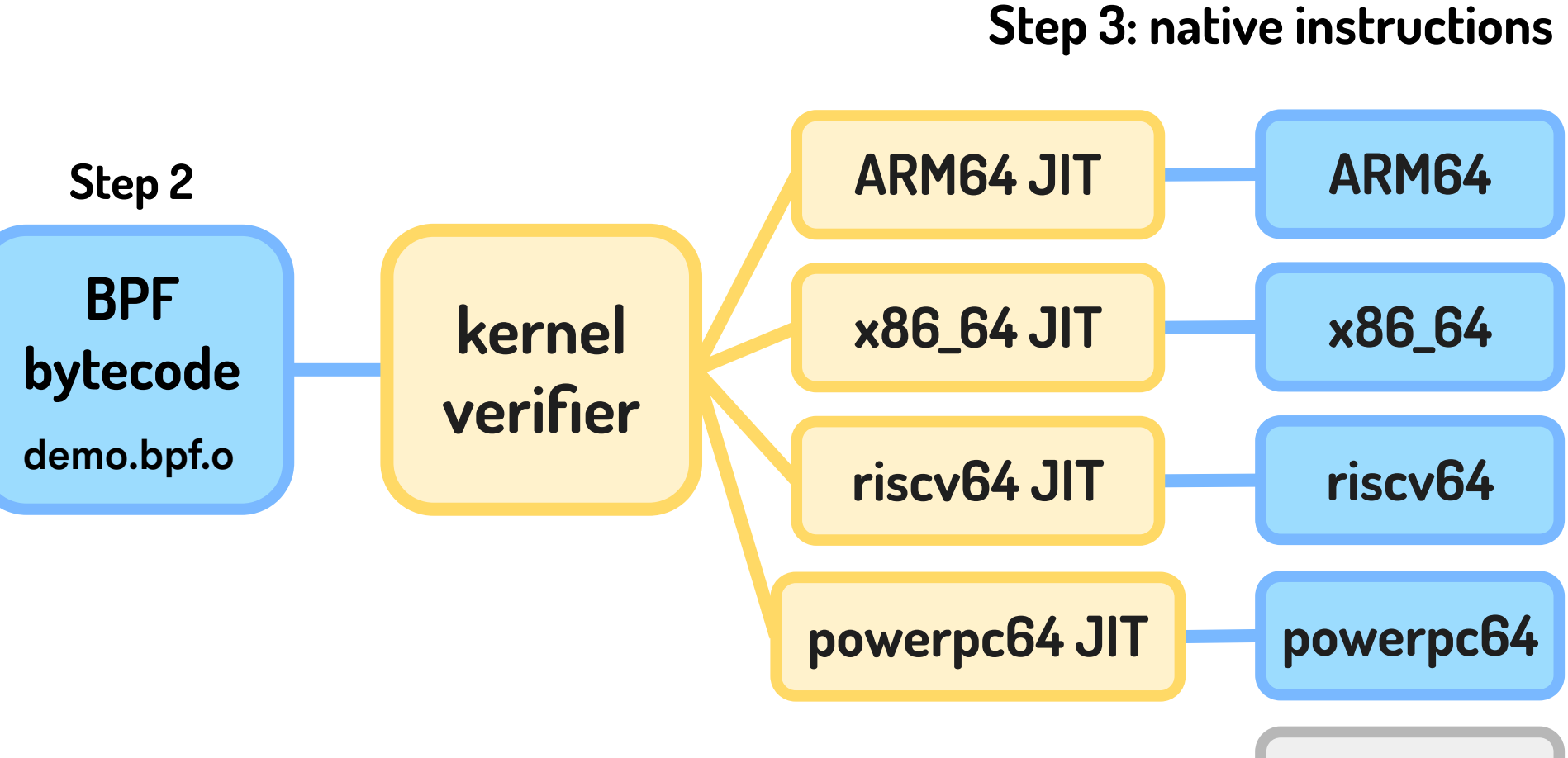
What about **sign-extending (sext)** ?

Sign extension, cont

```
/// Returns how the platform's atomic operations are
/// extended (ZERO_EXTEND, SIGN_EXTEND, or ANY_EXTEND).
virtual ISD::NodeType getExtendForAtomicOps() const {
  return ISD::ZERO_EXTEND;
}
```

- LLVM **expects** an architecture's (sub-word) atomic operations to be one of **zext**, **sext** or undefined ("**anyext**")
- Non-trivial to have both **zext** and **sext**; BPF uses the default (**zext**)


Step 3: native instructions
Step 1
C
demo.bpf.c
Step 2
BPF
bytecode
demo.bpf.o
ARM64
x86_64
riscv64
powerpc64


Step 3: native instructions
Step 2
BPF bytecode
demo.bpf.o
kernel verifier
ARM64 JIT
x86_64 JIT
riscv64 JIT
powerpc64 JIT
ARM64
x86_64
riscv64
powerpc64

Kernel BPF verifier

- Rejects **unaligned** load-acquire/store-release instructions, even if the **BPF_F_ANY_ALIGNMENT** load flag is set
- Rejects 64-bit load-acquire/store-release instructions on 32-bit architectures

Just-in-time (JIT) compilers, ARM64

- Emits **LDAR** for load-acquire, and **STRL** for store-release
- **LDAR** has **RCsc** semantics; the BPF load-acquire instruction is defined to have the weaker **RCpc** semantics (like ARM64 **LDAPR**)
  - Summary of discussions on BPF load-acquire instruction ordering
  - [PATCH RFC bpf-next v1 0/4] Introduce load-acquire and store-release BPF instructions, look for "**LDAPR**"

Just-in-time (JIT) compilers, x86_64

- Load-acquire **(LL + LS)** and store-release **(LS + SS)** requirements are already guaranteed by x86_64 **TSO**: **LL + LS + SS**
- Simply emits regular, aligned loads/stores
  - For example, **MOVZBQ** for 8-bit load-acquire

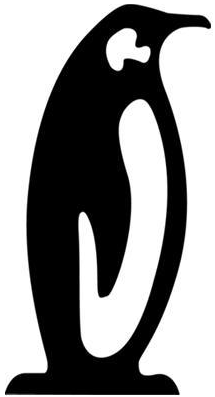
Next steps

- JIT support for other architectures
- explicit memory barriers using **bpf_fastcall kfuncs** (f.k.a. nocsr kfuncs)
  - like kernel **smp_mb()** and friends
- GCC support
- **__ATOMIC_SEQ_CST** not supported yet in LLVM
- BPF JIT compiler barrier
- make **-mcpu=auto** work for v4 instructions

Other references

- Kernel **Documentation/bpf/standardization/instruction-set.rst**
- Kernel **Documentation/memory-barriers.txt**
- SelectionDAG Instruction Selection Process
- Is Parallel Programming Hard, And, If So, What Can You Do About It?
- Initial proposal: Supporting New Memory Barrier Types in BPF
- LLVM commit 17bfc00f7c4a ("[BPF] Add load-acquire and store-release instructions under -mcpu=v4 (#108636)")
- Kernel commit 880442305a39 ("bpf: Introduce load-acquire and store-release instructions")
- SlidesCodeHighlighter

Thank you!



東京 2025 LINUX PLUMBERS CONFERENCE

TOKYO, JAPAN / DECEMBER 11-13, 2025